

Implementation and experimentation with Dynamic Staging

LP2, 2nd May 2026

CHING Long Tin, YEUNG Sin Chun

Motivation	1
Compile-Time vs. Run-Time Work	2
Literature Survey	4
Overview	13
Staging	26
Shape Propagation	35
Inlining	71
Printing Staged Block	74
Benchmarking	77
Bibliography	79

Compile-Time vs. Run-Time Work

Programmers often write code in a clear, general style, even when parts of it could be computed during compilation.

Original program

```
1 fun dot(xs, ys, i) = mls
2   if xs.length == i then 0
3   else xs.(i) * ys.(i)
4       + dot(xs, ys, i + 1)
5
6 fun dotWith3(v) =
7   dot([1, 0, 2], v, 0)
```

... can actually be written as

```
1 fun dotWith3(v) = mls
2   v(0) + 2 * v(2)
```

Compile-Time vs. Run-Time Work

The recursion, the pattern matching, the multiplication are all known at compile time. The residual program contains only the work that genuinely depends on the runtime input v .

Goal: let the programmer keep the abstract version, and have the compiler peel away the static layer automatically.

Motivation	1
Literature Survey	4
Multi-Stage Programming	5
Linking Rewriting DSLs	8
Class specialization	9
First Class Functions	11
Overview	13
Staging	26
Shape Propagation	35
Inlining	71
Printing Staged Block	74
Benchmarking	77
Bibliography	79

Multi-Stage Programming

Well-known optimization technique [1]

```
1 fun power(n, x) = if n is
2   0 then .<1>.
3   else .<~x * ~ (power(n - 1, x))>.
4 fun power2 = .! .<(x => ~ (power(2, .<x>.) ) )>.
```

In here, green annotates code to be executed in the next stage, and gray executed in the current stage.

Multi-Stage Programming

Rule for $.!$: evaluate until the whole code fragment is in the next stage, then move it to the current stage.


$$.!\left(\left(\left(X\right)\right)\right) = .!\left(X\right) = X$$

During evaluation, we are able to pre-compute certain parts of the code, reducing runtime.

Multi-Stage Programming

1 `x => power(2, x)`

mls

2 `x => if 2 is 0 then 1 else x * power(2 - 1, x)`

3 `x => x * power(1, x)`

4 `x => x * if 1 is 0 then 1 else x * power(1 - 1, x)`

5 `x => x * x * if 0 is 0 then 1 else x * power(0 - 1, x)`

6 `x => x * x * 1`

7 `fun power2 = x => x * x * 1`

Linking Rewriting DSLs

Improve the compilation of functions with domain-specific modules. [2]

Drawback: The annotations need to be manually added, and so the library designer needs to anticipate uses of the library.

Improvement: Code fragments are evaluated separately, when the result of a code fragment may be reused.

Class specialization

```
1 class B(val x) with
2   fun f() = ...
3 class D1(val x) extends B
4   with
5   if x is
6     1 then new D1(1)
7     2 then new D2(2)
8
9   x.f()
```

```
...
8   if x is mls
9     D1 then x.D1_f1()
10    D2 then x.D2_f1()
11    D3 then x.D3_f1()
```

Class specialization

Traditional Multi-Stage Programming tracks types.

Even if x has a known class shape, we cannot remove matching arms.

Specialization is done through typing, so we know that $x : B$, but we do not know which specific derived class of B is used.

```
1 data class Foo(x, y) extends Bar(x+1, y+1)
```

mls

```
2
```

```
3 if new Foo(1, 2) is
```

```
4   Bar(x, y) then ...
```

First Class Functions

First Class Functions

Under single inheritance, matching arms runs into problems. [3]

```
1 class Foo$1$2 extends Bar
2 class Bar$2$3
```

mls

Drawback: Either we perform class splitting, or we do not specialize inherited classes.

Motivation 1

Literature Survey 4

Overview 13

 High-Level Approach 14

 MLscript Compiler 17

 Dynamic Staging Recipe 18

Staging 26

Shape Propagation 35

Inlining 71

Printing Staged Block 74

Benchmarking 77

Bibliography 79

High-Level Approach

Start with an ordinary module:

```
1 module M with
2   fun dot(xs, ys, i) =
3     if xs.length == i then 0
4     else xs.(i) * ys.(i) + dot(xs, ys, i + 1)
5   fun dotWith3(v) = dot([1, 0, 2], v, 0)
```

mls

High-Level Approach

Users can mark a module as staged which turns the module into a **code generator**. When we execute the program, it emits a new residual module instead of executing the program.

```
1  staged module M with
2    fun dot(xs, ys, i) =
3      if xs.length == i then 0
4      else xs.(i) * ys.(i) + dot(xs, ys, i + 1)
5  fun dotWith3(v) = dot([1, 0, 2], v, 0)
```

mls

High-Level Approach

The output of executing the program is

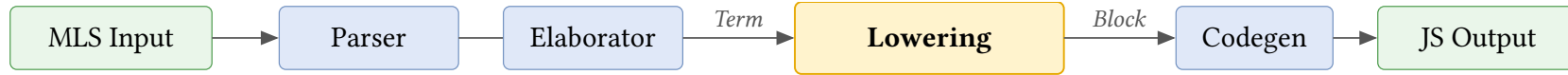
```
1 module M with
2   fun dotWith3(v) = v.(0) + 2 * v.(2)
3   // ... with more specialized functions ...
```

mls

which the user can import.

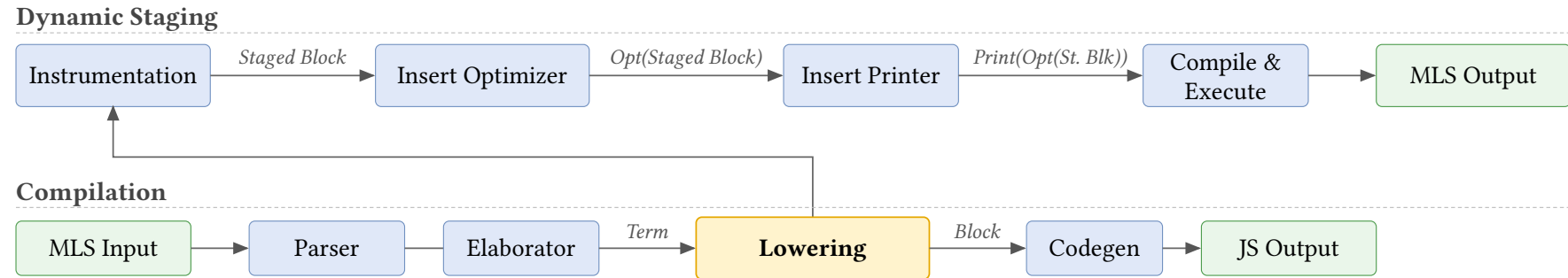
Hence, our approach is a combination of **metaprogramming** (code generator) and **specialization** (generation of specialized functions).

MLscript Compiler



- Lexer/Parser: converts source code into AST
- Elaborator/Resolver: Deal with references
- **Lowering**: converts AST to a simplified format
- Codegen: Turn AST into executable code (JavaScript).

Dynamic Staging Recipe

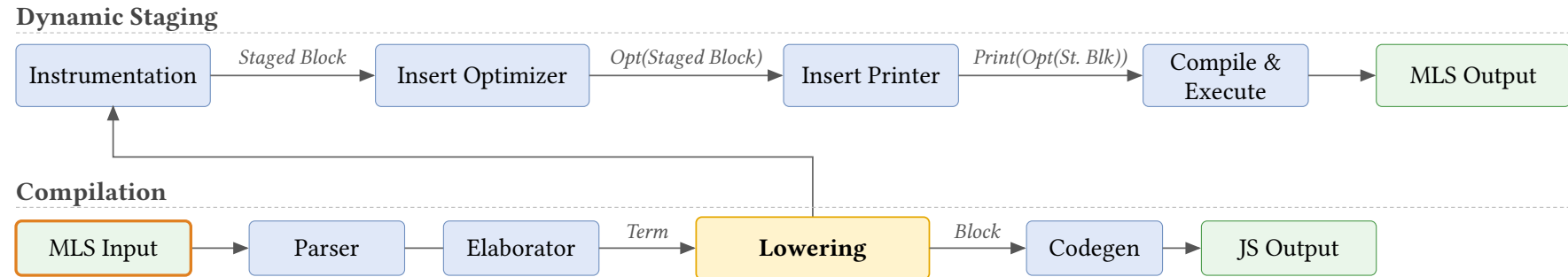


Two phases: Instrumentation / Insert Optimizer

Lowering Pass

We implement staging a transformation from Scala Block $\Rightarrow$ Scala Block.

Dynamic Staging Recipe

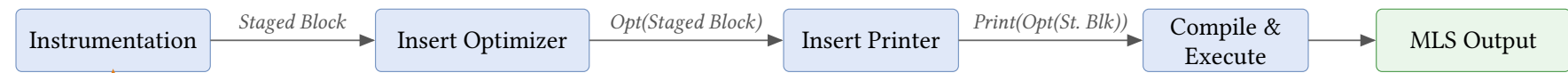


Source (MLscript)

```
if C(0) is mls
  Int then "Int"
  C(n) then "C(" + n + ")"
  else "Unknown"
```

Dynamic Staging Recipe

Dynamic Staging



Compilation



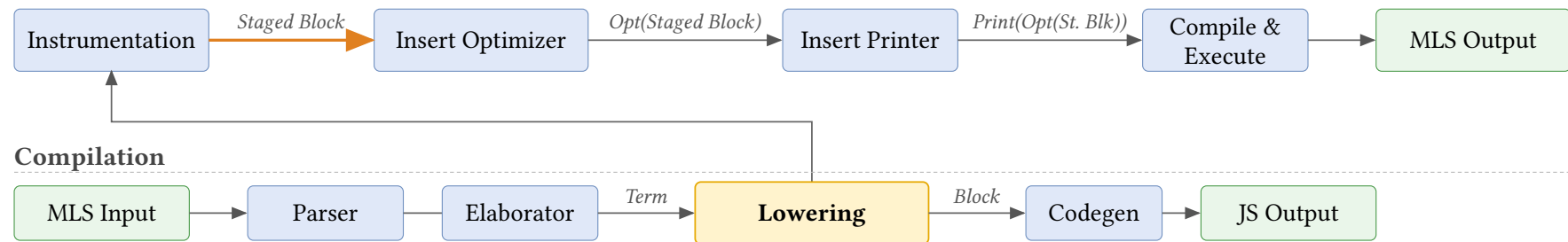
Block

```
Scoped([scrut, n, arg, tmp],  
  Assign(scrut, Call(C, [0])),  
  Match(Ref(scrut), [  
    Arm(Cls(Int), Ret("Int")),  
    Arm(Cls(C),  
      Assign(arg, Select(scrut, "n")),  
      Assign(n, Ref(arg)),  
      Assign(tmp, Call("+", ["C(", n])),  
      Ret(Call("+", [tmp, "]))  
    ], Ret("Unknown"))
```

Scala

Dynamic Staging Recipe

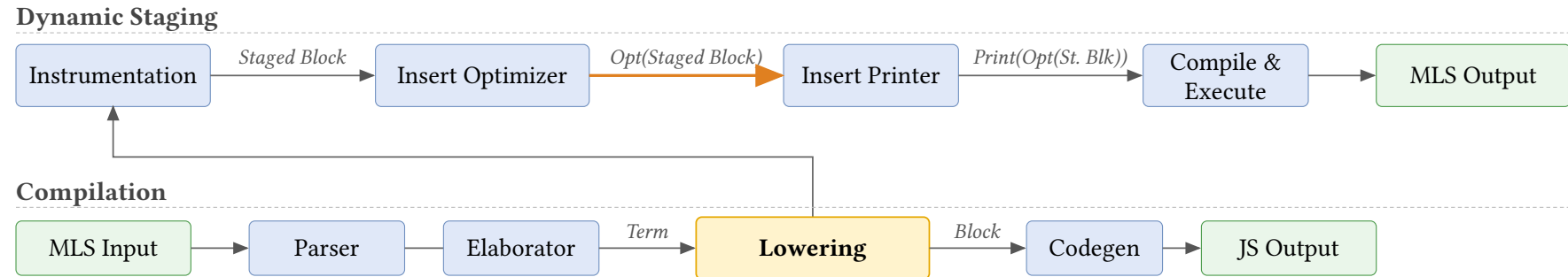
Dynamic Staging



Staged Block

```
Scoped([scrut, n, arg, tmp],  
  Assign(scrut, Call(C, [0])),  
  Match(Ref(scrut), [  
    Arm(Cls(Int), Ret("Int")),  
    Arm(Cls(C),  
      Assign(arg, Select(scrut, "n")),  
      Assign(n, Ref(arg)),  
      Assign(tmp, Call("+", ["C(", n])),  
      Ret(Call("+", [tmp, ")"]))  
    ], Ret("Unknown"))  
])
```

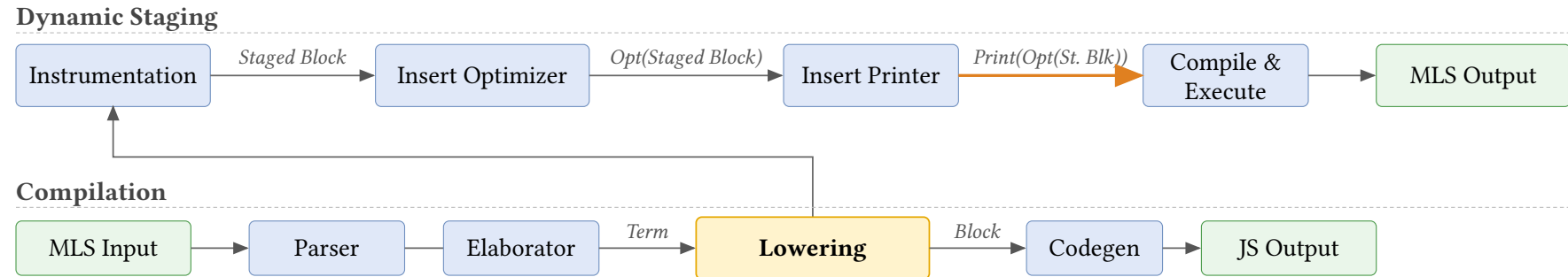
Dynamic Staging Recipe



Opt(Staged Block)

```
prop(
  Scoped([scrut, n, arg, tmp],
    Assign(scrut, Call(C, [0])),
    Match(Ref(scrut), [
      Arm(Cls(Int), Ret("Int")),
      Arm(Cls(C),
        Assign(arg, Select(scrut, "n")),
        Assign(n, Ref(arg)),
        Assign(tmp, Call("+", ["C(", n])),
        Ret(Call("+", [tmp, ""]))
      ], Ret("Unknown")))
  )
)
```

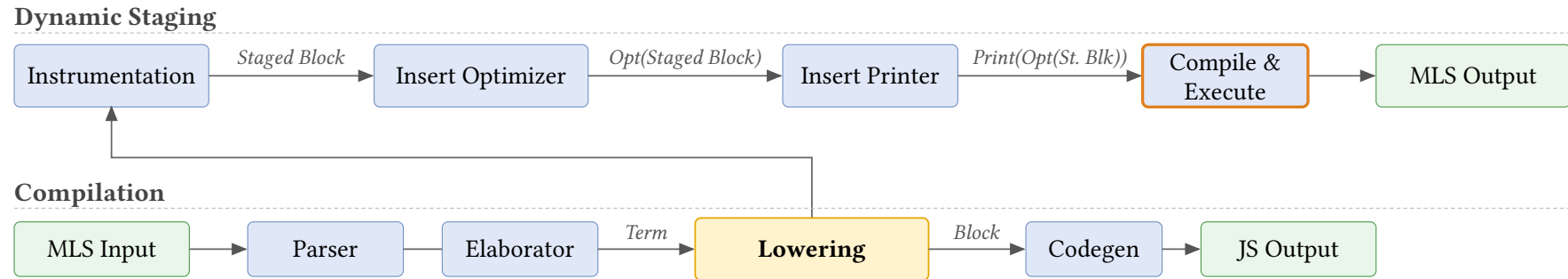
Dynamic Staging Recipe



Print(Opt(Staged Block))

```
print(  
  prop(  
    Scoped([scrut, n, arg, tmp],  
      Assign(scrut, Call(C, [0])),  
      Match(Ref(scrut), [  
        Arm(Cls(Int), Ret("Int")),  
        Arm(Cls(C),  
          Assign(arg, Select(scrut, "n")),  
          Assign(n, Ref(arg)),  
          Assign(tmp, Call("+", ["C(", n])),  
          Ret(Call("+", [tmp, ")"]))  
        ], Ret("Unknown"))  
      )  
    )  
  )  
)
```

Dynamic Staging Recipe

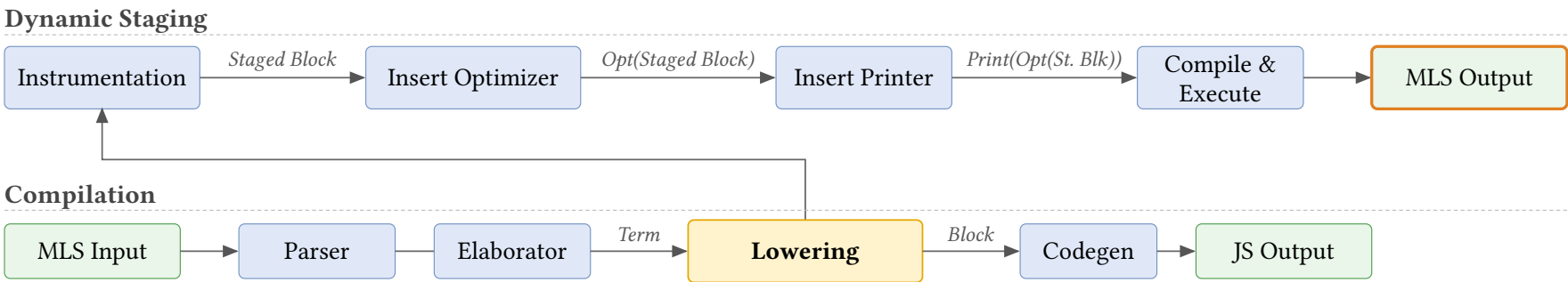


Compile & Execute

This execution is the **first stage** of metaprogramming. It entails the execution of

- ▶ the **optimizer** which outputs the **optimized staged block**,
- ▶ and the **printer** which prints the new MLscript program from the optimized staged block.

Dynamic Staging Recipe



Generated MLscript

```
"C(0)" mls
```

Motivation	1
Literature Survey	4
Overview	13
Staging	26
Function-to-Class Transformation	27
Staging Block	28
Staging Symbols	32
Instrumentation	33
Shape Propagation	35
Inlining	71
Printing Staged Block	74
Benchmarking	77
Bibliography	79

Function-to-Class Transformation

Simplify lambda functions by turning them into classes

Staging Block

```
1 case class Return(res, implct) extends Block
2 case class Match(scrut, arms, dflt, rest) extends Block
3 case class Assign(lhs, rhs, rest) extends Block
```

Scala

```
1 class Block with
2   constructor
3   Return(res, implct)
4   Match(scrut, arms, dflt, rest)
5   Assign(lhs, rhs, rest)
```

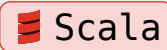
mls

For any Scala Block data, we can recreate the same structure within MLscript which we called **Staged Block**.

Staging Block

```
fun pow(x, n) = if n is 0 then 1 else x * pow(x, n-1)
```

```
1  val pow = Symbol("pow"); val x = Symbol("x"); val n = Symbol("n")
2  val t1 = Symbol("tmp"); val t2 = Symbol("tmp")
3  val sub = Symbol("-"); val mul = Symbol("*")
4  FunDefn(pow, [x, n], Scoped(HashSet(t1, t2), Match(Ref(n),
5    [[ Lit(0), Return(Lit(1)) ]],
6    Assign(t1, Call(sub, [n, Lit(1)]),
7    Assign(t2, Call(pow, [x, t1]),
8    Return(Call(mul, [x, t2]))
9    )
10  ), End())
11  ))
```



As long as we have the corresponding constructor, we can copy over the structure to the Staged Block.

Staging Block

```
fun pow(x, n) = if n is 0 then 1 else x * pow(x, n-1)
```

```
1  let pow = Symbol("pow"); let x = Symbol("x"); let n = Symbol("n")
2  let t1 = Symbol("tmp"); let t2 = Symbol("tmp")
3  let sub = Symbol("-"); let mul = Symbol("*")
4  FunDefn(pow, [x, n], Scoped([t1, t2], Match(Ref(n),
5    [[ Lit(0), Return(Lit(1)) ]],
6    Assign(t1, Call(sub, [n, Lit(1)]),
7    Assign(t2, Call(pow, [x, t1]),
8    Return(Call(mul, [x, t2]))
9    )
10  ), End())
11  ))
```

mls

As long as we have the corresponding constructor, we can copy over the structure to the Staged Block.

Staging Block

We perform structural induction to stage each type of Scala Block.

`Value.Lit(lit)`

`ValueLit(lit)`

mls

Example: $1 \mapsto \text{ValueLit}(1)$

`Symbol(name)`

`Symbol(name)`

mls

Example:

- $x \mapsto \text{Symbol}("x")$
- $C \mapsto \dots?$

This is not enough for staging symbols. We'll revisit this case later on.

Staging Block

We perform structural induction to stage each type of Scala Block.

`Value.Lit(lit)`

`ValueLit(lit)`

`mls`

Example: $1 \mapsto \text{ValueLit}(1)$

`Value.Ref(sym)`

`let l = sym`

`mls`

`ValueRef(l)`

Example: $x \mapsto \text{ValueRef}(\text{Symbol}("x"))$

`Select(qual, name)`

`let q = qual`

`mls`

`let n = name`

`Select(q, n)`

Example: $x.p \mapsto \text{Select}(\text{ValueRef}(\text{Symbol}("x")), \text{Symbol}("p"))$

Staging Block

```
Tuple([x0, x1, ...])
```

```
let s0 = x0
```

mls

```
let s1 = x1
```

...

```
Tuple([x0, x1, ...])
```

The other Scala Block cases are similar, staging the parameters of the Scala Block by induction and recreating the corresponding Staged Block counterpart.

Staging Symbols

- ▶ We need more information about the Symbol in the original stage

Add reference to current value in the symbol

C.x

```
1 let CSym = ClassSymbol("C", C)
2 Select(ValueRef(CSym), Symbol("x"))
```

mls

Add redirection within current stage to allow access for next stage

```
1 staged class A with
2   fun f() = M.f()
3   val redirect_M = M
```

mls

Staging Symbols

- Uniqueness of Symbols

We want Symbols for the same object in the previous stage to be unique in the current stage across functions and compilations.

For local symbols, we can maintain a map during staging to reuse a staged symbol.

`x + x`

```
1 let x = Symbol("x")  
2 // let x1 = Symbol("x")  
3 Call(ValueRef(Symbol("+")), [[ValueRef(x), ValueRef(x)]])
```

mls

Staging Symbols

- Uniqueness of Symbols

We want Symbols for the same object in the previous stage to be unique in the current stage across functions and compilations.

For class and module symbols, we need to cache and use first instance of a symbol within the staged code.

`M.f()`

```
1 let MSym = symbolMap.check(ModuleSymbol("M", M))
2 Call(Select(ValueRef(MSym), Symbol("f")), [[]])
```

mls

Instrumentation

Insert some auxiliary helper variables/functions to the module for shape propagation.

```
1  staged module Math with
2    val funCache = new Map()
3    val generatorMap = new Map([["pow", pow_gen]])
4    fun pow_staged() = ...
5    fun pow_gen(x, n) =
6      specialize(funCache, "f", pow_staged, [x, n])
7    fun propagate() =
8      let dyn = ...
9      pow_gen(dyn, dyn)
10     sq_gen(dyn)
```

mls

Instrumentation

Insert some auxiliary helper variables/functions to the module for shape propagation.

Memoization is based on [4].

```
1 staged module Math with
2   val funCache // memoize specialized functions
3   val generatorMap // point to generator function
4   fun pow_staged() // return staged version of the function
5   fun pow_gen(x, n) // redirect to shape propagation
6   fun propagate() // generates all entry functions
```

mls

Instrumentation

For staged classes, we can treat the parameters of the class as the function parameter, then specialize the functions as usual.

```
1 staged class C(x, y) with  
2   fun add() = x + y
```

mls

```
1 staged module C with  
2   fun add(cls)() = cls.x + cls.y
```

mls

Motivation	1
Literature Survey	4
Overview	13
Staging	26
Shape Propagation	35
Block in BNF	37
Shape Definition	38
Shape of Path ($\Gamma \vdash p \Rightarrow s$)	40
Shape of Result ($\Gamma \vdash r \rightsquigarrow r' \Rightarrow s$)	42
Shape Propagation	49
Pattern Matching (Formalization)	54
Staged Class	63
Inlining	71
Printing Staged Block	74

Benchmarking	77
Bibliography	79

Block in BNF

Symbol x, y, z

Literal ι

Function definition **fun** $f(\overline{x_i^n}) = b$

Plain class definition **class** $C(\overline{\text{val } n})$

Staged module definition **staged module** M **with fun** $f(\overline{x_i^n}) = b$

Path $p ::= \iota \mid x \mid p.n$

Result $r ::= p \mid f(\overline{p}) \mid \text{new } C(\overline{p}) \mid [\overline{p}]$

Match pattern $\pi ::= \iota \mid C \mid []^n \mid _$

Match arm $\kappa ::= \pi \text{ then } b$

Non-tail block $B ::= \varepsilon \mid \text{if } p \text{ is } \overline{\kappa}; B \mid x = r; B \mid \text{let } x; B$

Block $b ::= \text{return } r \mid \text{end} \mid B; b$

Shape Definition

A **Shape** captures what we statically know about a value at compile time.

$$\text{Shape } s ::= \iota \mid \mathbf{dyn} \mid [\bar{s}] \mid C(\overline{n : s}) \mid \perp \mid s \cup s$$

We write $\llbracket s \rrbracket$ to denote the shape of an expression, distinguishing it from actual values.

```
1 fun f(p, cond) =
```

mls

```
2   let a = 42
```

$\llbracket 42 \rrbracket$

```
3   let b = C(p, 2)
```

$\llbracket C(\text{dyn}, 2) \rrbracket$

```
4   let c = if cond then [1, 2] else C(1, 2)
```

$\llbracket [1, 2] \cup C(1, 2) \rrbracket$

```
5   let d = if b is C then 0 else 1
```

$\llbracket 0 \rrbracket$

The mechanism to calculate the shapes are called **Shape Propagation**.

Shape Definition

A Tiny Example

```
1 staged module M with
2   fun add1(x) = x + 1
3   fun test()  = add1(2)
```

mls

Walking through `test()`:

1. Argument `x` has shape $\llbracket 2 \rrbracket$ (a literal).
2. Specialise `add1` with $x \mapsto \llbracket 2 \rrbracket$ inside the body, `x` looks up $\llbracket 2 \rrbracket$ in the context.
3. `x + 1` folds: $\llbracket 2 \rrbracket + \llbracket 1 \rrbracket \rightarrow \llbracket 3 \rrbracket$.
4. Cache the result as `add1_Lit2() = 3`; rewrite the call site `add1(2)` to `add1_Lit2()`.

Final shape of `test()`: $\llbracket 3 \rrbracket$. Body collapses to a constant.

Shape of Path ($\Gamma \vdash p \Rightarrow s$)

We evaluate paths p to their shapes s .

$$\frac{}{\Gamma \vdash \iota \Rightarrow \llbracket \iota \rrbracket}$$

Example

42

mls

$\Gamma = \varepsilon$

$\varepsilon \vdash 42 \Rightarrow \llbracket 42 \rrbracket$

$$\frac{(p \mapsto s) \in \Gamma \quad s \neq \perp}{\Gamma \vdash p \Rightarrow s}$$

Example

x

mls

$\Gamma = x \mapsto \llbracket C(n : \llbracket 1 \rrbracket) \rrbracket$

$\Gamma \vdash x \Rightarrow \llbracket C(n : \llbracket 1 \rrbracket) \rrbracket$

$$\frac{p \notin \text{dom}(\Gamma) \quad \Gamma \vdash p \Rightarrow s}{\Gamma \vdash p.n \Rightarrow \text{sel}(s, \llbracket n \rrbracket)}$$

Example

x.n

mls

$\Gamma = x \mapsto \llbracket C(n : \llbracket 1 \rrbracket) \rrbracket$

$\Gamma \vdash x.n \Rightarrow \llbracket 1 \rrbracket$

$$p ::= \iota \mid x \mid p.n$$

Shape of Path ($\Gamma \vdash p \Rightarrow s$)

Shape Selection (**sel**)

sel computes the shape of a selection.

$$\begin{aligned} \text{sel}(\llbracket C(\overline{n_i : s_i}) \rrbracket, \llbracket n_i \rrbracket) &= s_i & \text{sel}(\mathbf{dyn}, \llbracket n_i \rrbracket) &= \mathbf{dyn} \\ \text{sel}(\llbracket [\overline{s_i}^{i \in 0..n}] \rrbracket, \llbracket i \rrbracket) &= s_i & \text{sel}(s_1 \cup s_2, s) &= \text{sel}(s_1, s) \cup \text{sel}(s_2, s) \\ \text{sel}(s_1, s_2) &= \mathbf{err} \end{aligned}$$

Example

$$\begin{aligned} \text{sel}(\llbracket [1, 2] \rrbracket \cup \llbracket [2, 3, 4] \rrbracket, \llbracket 1 \rrbracket) &= \text{sel}(\llbracket [1, 2] \rrbracket, \llbracket 1 \rrbracket) \cup \text{sel}(\llbracket [2, 3, 4] \rrbracket, \llbracket 1 \rrbracket) \\ &= \llbracket 2 \rrbracket \cup \llbracket 3 \rrbracket \end{aligned}$$

Shape of Result ($\Gamma \vdash r \rightsquigarrow r' \Rightarrow s$)

We optimize block and track shapes simultaneously: r becomes r' with shape s .

$$\frac{\Gamma \vdash p \Rightarrow s}{\Gamma \vdash p \rightsquigarrow p \Rightarrow s}$$

Example

`x` mls

$$\begin{aligned}\Gamma &= x \mapsto \llbracket 1 \rrbracket \\ \Gamma \vdash x \rightsquigarrow x &\Rightarrow \llbracket 1 \rrbracket\end{aligned}$$

$$\frac{\Gamma \vdash p_i \Rightarrow s_i}{\Gamma \vdash [\overline{p_i^n}] \rightsquigarrow [\overline{p_i^n}] \Rightarrow \llbracket [\overline{s_i^n}] \rrbracket}$$

Example

`[x, y]` mls

$$\begin{aligned}\Gamma &= x \mapsto \llbracket 1 \rrbracket, y \mapsto \llbracket 2 \rrbracket \\ \Gamma \vdash [x, y] \rightsquigarrow [x, y] &\Rightarrow \llbracket \llbracket 1 \rrbracket, \llbracket 2 \rrbracket \rrbracket\end{aligned}$$

$$\frac{\Gamma \vdash p_i \Rightarrow s_i}{\Gamma \vdash \mathbf{new} C(\overline{p_i}) \rightsquigarrow \mathbf{new} C(\overline{p_i}) \Rightarrow \llbracket C(\overline{n_i} : \overline{s_i}) \rrbracket}$$

Example

`new Pair(x, y)` mls

$$\begin{aligned}\Gamma &= x \mapsto \llbracket 1 \rrbracket, y \mapsto \llbracket 2 \rrbracket \\ \Gamma \vdash \mathbf{new} \text{Pair}(x, y) &\Rightarrow \llbracket \text{Pair}(a : \llbracket 1 \rrbracket, b : \llbracket 2 \rrbracket) \rrbracket\end{aligned}$$

$$r ::= p \mid [\overline{p}] \mid \mathbf{new} C(\overline{p}) \mid f(\overline{p})$$

Shape of Result ($\Gamma \vdash r \rightsquigarrow r' \Rightarrow s$)

Shape of Result (sor) – Staged Application

When f is **staged**, we recursively specialise its body on the argument shapes.

$$\frac{f \text{ is staged} \quad \Gamma \vdash p_i \Rightarrow s_i \quad f_{\text{gen}}(\bar{p}, \bar{s}) = (r', s)}{\Gamma \vdash f(\bar{p}_i) \rightsquigarrow r' \Rightarrow s}$$

Example

```
staged module Staged with
  fun pyth(x, y) = x * x + y * y
  fun test() = pyth(2, 3)
```

mls

Specialise $\text{pyth}_{\text{gen}}(\llbracket 2 \rrbracket, \llbracket 3 \rrbracket) \rightarrow \text{pyth_Lit2_Lit3}() = 13$, shape $\llbracket 13 \rrbracket$. The specialized function will be saved to the cache.

Conclusion: $\varepsilon \vdash \text{pyth}(2, 3) \rightsquigarrow 13 \Rightarrow \llbracket 13 \rrbracket$

$$r ::= p \mid [\bar{p}] \mid \text{new } C(\bar{p}) \mid f(\bar{p})$$

Shape of Result ($\Gamma \vdash r \rightsquigarrow r' \Rightarrow s$)

Shape of Result (sor) – Staged Application

After propagation, the call site `pyth(2, 3)` is rewritten to its cached variant.

Example

```
staged module Staged with
  fun pyth_Lit2_Lit3() = 13
  fun test() = pyth_Lit2_Lit3()
```

mls

Cache

Function	Shapes	Symbol	Block	Returned Shape
pyth	(<code>[[2]]</code> , <code>[[3]]</code>)	pyth_Lit2_Lit3	return 13	<code>[[13]]</code>
pyth	(<code>dyn</code> , <code>dyn</code>)	pyth_Dyn_Dyn	return <code>x*x+y*y</code>	<code>dyn</code>

If f_{gen} is called with the same $(f, \bar{s})$ again, it looks up the existing entry in the cache instead of re-specialising.

Shape of Result ($\Gamma \vdash r \rightsquigarrow r' \Rightarrow s$)

Shape of Result (sor) – Non-Staged Application

When f is **non-staged**, we don't have its Staged Block. Two sub-cases.

$$\frac{f \text{ is non-staged} \quad \Gamma \vdash p_i \Rightarrow s_i \quad \forall i. \text{static}(s_i) \quad f_{\text{imp}}(\overline{\text{valOf}(s)}) = (r, s)}{\Gamma \vdash f(\overline{p_i}) \rightsquigarrow r \Rightarrow s}$$

Example

```
NonStaged.sq(2)
```

```
mls
```

All args static; evaluate $\text{sq}(2) = 4$.

$\varepsilon \vdash \text{sq}(2) \rightsquigarrow 4 \Rightarrow \llbracket 4 \rrbracket$

$$\frac{f \text{ is non-staged} \quad \Gamma \vdash p_i \Rightarrow s_i \quad \exists i. \neg \text{static}(s_i)}{\Gamma \vdash f(\overline{p_i}) \rightsquigarrow f(\overline{p_i}) \Rightarrow \mathbf{dyn}}$$

Example

```
// where x is dynamic
```

```
mls
```

```
NonStaged.sq(x)
```

$\Gamma = x \mapsto \mathbf{dyn}$; cannot evaluate, call remain unchanged.

$\Gamma \vdash \text{sq}(x) \rightsquigarrow \text{sq}(x) \Rightarrow \mathbf{dyn}$

$$r ::= p \mid [\overline{p}] \mid \mathbf{new} \ C(\overline{p}) \mid f(\overline{p})$$

Shape of Result ($\Gamma \vdash r \rightsquigarrow r' \Rightarrow s$)

Non-Staged Application Example

```
module NonStaged with mls  
  fun sq(x) = x * x  
  
  staged module Staged with  
    fun pyth(x, y) =  
      NonStaged.sq(x) +  
      NonStaged.sq(y)  
  fun test() = pyth(2, 3)
```

After staging:

```
module Staged with mls  
  fun pyth_Lit2_Lit3() = 13  
  fun test() = 13
```

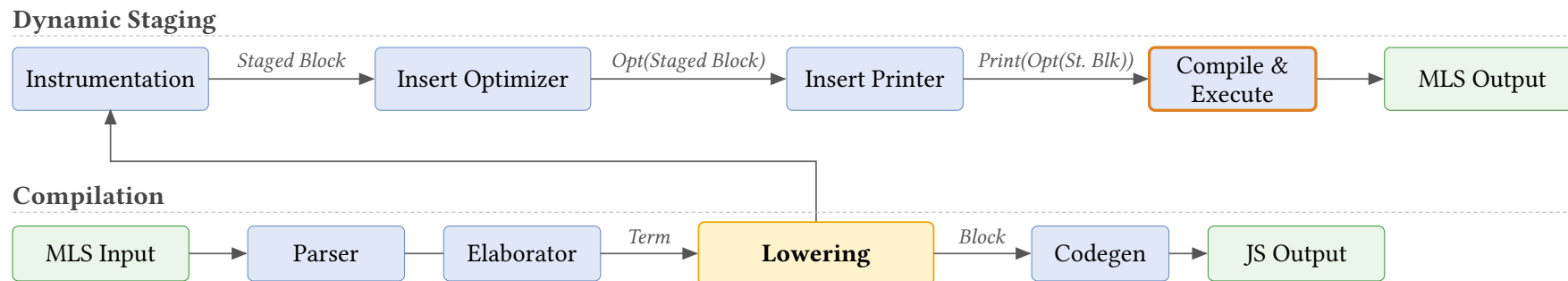
- ▶ Since every params of `sq(x)` is static, we will call `sq` to evaluate the result completely.
- ▶ No specialised functions are generated for the non-staged module `NonStaged`.

Shape of Result ($\Gamma \vdash r \rightsquigarrow r' \Rightarrow s$)

How do we call the function `NonStaged.sq`?

Recall that during execution of the optimizer, we already compile the function `NonStaged.sq(x)` to JavaScript.

- ▶ Essentially we get the optimization for free in the static parameter case (at no expense of code size)



Shape of Result ($\Gamma \vdash r \rightsquigarrow r' \Rightarrow s$)

Static Shape

$$\text{static}(\llbracket \iota \rrbracket) = \mathbf{true}$$

$$\text{static}(\llbracket [\overline{s_i^n}] \rrbracket) = \bigwedge_i \text{static}(s_i)$$

$$\text{static}(\llbracket C(\overline{n_i : s_i}) \rrbracket) = \bigwedge_i \text{static}(s_i) \quad \text{static}(s_1 \cup s_2) = \text{static}(s_1) \wedge \text{static}(s_2)$$

$$\text{static}(\mathbf{dyn}) = \mathbf{false}$$

$$\text{static}(\perp) = \mathbf{false}$$

Remark. $\text{static}(s)$ returns true iff no subshape of s contains **dyn**.

$$r ::= p \mid [\overline{p}] \mid \mathbf{new} \ C(\overline{p}) \mid f(\overline{p})$$

Shape Propagation

Pattern Matching

Before giving the formal rules, let's give a simple trace of shape propagation for pattern matching.

```
1  class C(val n)
2  staged module Simple with
3    fun f(x) =
4      let y
5        if x is C then y = x.n
6        else y = 0
7      y + 1
8  fun test()      = f(C(2))
9  fun test2(dyn) = f(C(dyn))
10 fun test3()     = f(0)
```

mls

Shape Propagation

Trace 1: test() = f(C(2))

Specialise f with $x \mapsto \llbracket C(n: 2) \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket C(n: 2) \rrbracket$

Shape Propagation

Trace 1: `test() = f(C(2))`

Specialise `f` with $x \mapsto \llbracket C(n: 2) \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket C(n: 2) \rrbracket$

$y \mapsto \llbracket \perp \rrbracket$

1. let `y`: extend ctx with $y \mapsto \llbracket \perp \rrbracket$

Shape Propagation

Trace 1: `test() = f(C(2))`

Specialise `f` with $x \mapsto \llbracket C(n: 2) \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket C(n: 2) \rrbracket$

$y \mapsto \llbracket \perp \rrbracket$

1. let `y`: extend ctx with $y \mapsto \llbracket \perp \rrbracket$
2. if `x` is `C`: `filter( $\llbracket C(n: 2) \rrbracket$ , C) =  $\llbracket C(n: 2) \rrbracket$` so then is viable; rest = $\llbracket \perp \rrbracket$ so else is dead

Shape Propagation

Trace 1: `test() = f(C(2))`

Specialise `f` with $x \mapsto \llbracket C(n: 2) \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket C(n: 2) \rrbracket$

$y \mapsto \llbracket 2 \rrbracket$

1. let `y`: extend ctx with $y \mapsto \llbracket \perp \rrbracket$
2. if `x` is `C`: `filter( $\llbracket C(n: 2) \rrbracket$ , C) =  $\llbracket C(n: 2) \rrbracket$  so then is viable; rest =  $\llbracket \perp \rrbracket$  so else is dead`
3. In then: `sop(x.n) = sel( $\llbracket C(n: 2) \rrbracket$ , n) =  $\llbracket 2 \rrbracket$ , so  $y \mapsto \llbracket 2 \rrbracket$`

Shape Propagation

Trace 1: `test() = f(C(2))`

Specialise `f` with $x \mapsto \llbracket C(n: 2) \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket C(n: 2) \rrbracket$

$y \mapsto \llbracket 2 \rrbracket$

1. let `y`: extend ctx with $y \mapsto \llbracket \perp \rrbracket$
2. if `x` is `C`: `filter`($\llbracket C(n: 2) \rrbracket$, `C`) = $\llbracket C(n: 2) \rrbracket$ so then is viable; rest = $\llbracket \perp \rrbracket$ so else is dead
3. In then: `sop`(`x.n`) = `sel`($\llbracket C(n: 2) \rrbracket$, `n`) = $\llbracket 2 \rrbracket$, so $y \mapsto \llbracket 2 \rrbracket$
4. `y + 1` folds to $\llbracket 3 \rrbracket$, then Return 3

Cached as `f_C_Lit2`; body folds entirely to the constant 3.

Shape Propagation

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: $\llbracket C(n: \text{dyn}) \rrbracket$.

Specialise `f` with $x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$.

ctx

$x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

Shape Propagation

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: $\llbracket C(n: \text{dyn}) \rrbracket$.

Specialise `f` with $x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$

$y \mapsto \llbracket \perp \rrbracket$

1. let `y: y` $\mapsto \llbracket \perp \rrbracket$

Shape Propagation

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: $\llbracket C(n: \text{dyn}) \rrbracket$.

Specialise `f` with $x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$

$y \mapsto \llbracket \perp \rrbracket$

1. let $y: y \mapsto \llbracket \perp \rrbracket$
2. if x is C : $\text{filter}(\llbracket C(n: \text{dyn}) \rrbracket, C) = \llbracket C(n: \text{dyn}) \rrbracket$ so then is viable; rest = $\llbracket \perp \rrbracket$ so else is dead

Shape Propagation

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: $\llbracket C(n: \text{dyn}) \rrbracket$.

Specialise `f` with $x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$

$y \mapsto \llbracket \text{dyn} \rrbracket$

1. let $y: y \mapsto \llbracket \perp \rrbracket$
2. if x is C : $\text{filter}(\llbracket C(n: \text{dyn}) \rrbracket, C) = \llbracket C(n: \text{dyn}) \rrbracket$ so then is viable; rest = $\llbracket \perp \rrbracket$ so else is dead
3. In then: $\text{sop}(x.n) = \text{sel}(\llbracket C(n: \text{dyn}) \rrbracket, n) = \llbracket \text{dyn} \rrbracket$

Shape Propagation

Trace 2: test2(dyn) = f(C(dyn))

Argument shape: $\llbracket C(n: \text{dyn}) \rrbracket$.

Specialise f with $x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$

$y \mapsto \llbracket \text{dyn} \rrbracket$

1. let $y: y \mapsto \llbracket \perp \rrbracket$
2. if x is C : $\text{filter}(\llbracket C(n: \text{dyn}) \rrbracket, C) = \llbracket C(n: \text{dyn}) \rrbracket$ so then is viable; $\text{rest} = \llbracket \perp \rrbracket$ so else is dead
3. In then: $\text{sop}(x.n) = \text{sel}(\llbracket C(n: \text{dyn}) \rrbracket, n) = \llbracket \text{dyn} \rrbracket$
4. $y + 1$: cannot fold ($\llbracket \text{dyn} \rrbracket + 1$); emit code, result $\llbracket \text{dyn} \rrbracket$

Cached as f_C_Dyn ; body keeps the $y = x.n$ assignment.

Shape Propagation

Trace 3: test3() = f(0)

Argument shape: $\llbracket 0 \rrbracket$ (literal).

Specialise f with $x \mapsto \llbracket 0 \rrbracket$.

ctx

$x \mapsto \llbracket 0 \rrbracket$

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

Shape Propagation

Trace 3: test3() = f(0)

Argument shape: $\llbracket 0 \rrbracket$ (literal).

Specialise f with $x \mapsto \llbracket 0 \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket 0 \rrbracket$

$y \mapsto \llbracket \perp \rrbracket$

1. let $y: y \mapsto \llbracket \perp \rrbracket$

Shape Propagation

Trace 3: `test3() = f(0)`

Argument shape: $\llbracket 0 \rrbracket$ (literal).

Specialise `f` with $x \mapsto \llbracket 0 \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket 0 \rrbracket$

$y \mapsto \llbracket \perp \rrbracket$

1. let $y: y \mapsto \llbracket \perp \rrbracket$
2. if x is C : $\text{filter}(\llbracket 0 \rrbracket, C) = \llbracket \perp \rrbracket$ so
then is dead; $\text{rest}(\llbracket 0 \rrbracket, C) = \llbracket 0 \rrbracket$ so
else is viable

Shape Propagation

Trace 3: `test3() = f(0)`

Argument shape: $\llbracket 0 \rrbracket$ (literal).

Specialise `f` with $x \mapsto \llbracket 0 \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket 0 \rrbracket$

$y \mapsto \llbracket 0 \rrbracket$

1. let $y: y \mapsto \llbracket \perp \rrbracket$
2. if x is C : $\text{filter}(\llbracket 0 \rrbracket, C) = \llbracket \perp \rrbracket$ so
then is dead; $\text{rest}(\llbracket 0 \rrbracket, C) = \llbracket 0 \rrbracket$ so
else is viable
3. In else: $y \mapsto \llbracket 0 \rrbracket$

Shape Propagation

Trace 3: `test3() = f(0)`

Argument shape: $\llbracket 0 \rrbracket$ (literal).

Specialise `f` with $x \mapsto \llbracket 0 \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket 0 \rrbracket$

$y \mapsto \llbracket 0 \rrbracket$

1. let $y: y \mapsto \llbracket \perp \rrbracket$
2. if x is C : $\text{filter}(\llbracket 0 \rrbracket, C) = \llbracket \perp \rrbracket$ so
then is dead; $\text{rest}(\llbracket 0 \rrbracket, C) = \llbracket 0 \rrbracket$ so
else is viable
3. In else: $y \mapsto \llbracket 0 \rrbracket$
4. $y + 1$ folds to $\llbracket 1 \rrbracket$, then Return 1

Cached as `f_Lit0`.

Shape Propagation

Resulting Cache

After all three calls, the staged module contains:

```
1  module Simple withs
2    fun f_C_Dyn(x) =           // remove match
3      let y
4        y = x.n
5        y + 1
6    fun f_C_Lit2(x) = 3        // fully specialised
7    fun f_Lit0() = 1          // fully specialised
8    fun test() = 3
9    fun test2(dyn) = f_C_Dyn(C(dyn))
10   fun test3() = 1
```

mls

Pattern Matching (Formalization)

Branch Filter (**filter**)

filter removes impossible shapes from the branches of a pattern match.

$$\begin{aligned} \text{filter}(s, _) &= s & \text{filter}(\llbracket \iota_1 \rrbracket, \iota_2) &= \llbracket \iota_1 \rrbracket \quad \text{if } \iota_1 = \iota_2 \\ \text{filter}(\llbracket \overline{s_i^n} \rrbracket, \llbracket _ \rrbracket^n) &= \llbracket \overline{s_i^n} \rrbracket & \text{filter}(\llbracket C(\overline{n_i : s_i^m}) \rrbracket, C) &= \llbracket C(\overline{n_i : s_i^m}) \rrbracket \\ \text{filter}(\mathbf{dyn}, \pi) &= \text{silh}(\pi) & \text{filter}(s_1 \cup s_2, \pi) &= \text{filter}(s_1, \pi) \cup \text{filter}(s_2, \pi) \\ \text{filter}(s, \pi) &= \perp \quad \text{otherwise} \end{aligned}$$

Example

$$\begin{aligned} \text{filter}(\llbracket C(n : \mathbf{dyn}) \rrbracket, C) &= \llbracket C(n : \mathbf{dyn}) \rrbracket \\ \text{filter}(\llbracket 0 \rrbracket, C) &= \perp \\ \text{filter}(\llbracket C(n : \mathbf{dyn}) \rrbracket \cup \llbracket 0 \rrbracket, C) &= \text{filter}(\llbracket C(n : \mathbf{dyn}) \rrbracket, C) \cup \text{filter}(\llbracket 0 \rrbracket, C) \\ &= \llbracket C(n : \mathbf{dyn}) \rrbracket \cup \perp = \llbracket C(n : \mathbf{dyn}) \rrbracket \end{aligned}$$

Pattern Matching (Formalization)

Silhouette Function (**silh**)

$\text{silh}(\pi)$ creates the shape of a pattern π , with all sub-shapes set to **dyn**.

$$\text{silh}(\iota) = \llbracket \iota \rrbracket \quad \text{silh}(C) = \llbracket C(\overline{\mathbf{dyn}}^n) \rrbracket \quad \text{silh}([\]^n) = \llbracket [\overline{\mathbf{dyn}}^n] \rrbracket \quad \text{silh}(_) = \mathbf{dyn}$$

Used in $\text{filter}(\mathbf{dyn}, \pi) = \text{silh}(\pi)$: when the scrutinee is **dyn**, the branch becomes viable with the silhouette shape — a concrete class/literal shell wrapping **dyn** sub-shapes.

Pattern Matching (Formalization)

Branch Remainder (rest)

rest removes shapes already covered by a match pattern.

$$\begin{aligned} \text{rest}(s, _) &= \perp & \text{rest}(\llbracket \iota_1 \rrbracket, \iota_2) &= \perp \quad \text{if } \iota_1 = \iota_2 \\ \text{rest}(\llbracket \overline{s_i^n} \rrbracket, \llbracket n \rrbracket) &= \perp & \text{rest}(\llbracket C(\overline{n_i : s_i^m}) \rrbracket, C) &= \perp \\ \text{rest}(\mathbf{dyn}, \pi) &= \mathbf{dyn} \quad \text{if } \pi \neq _ & \text{rest}(s_1 \cup s_2, \pi) &= \text{rest}(s_1, \pi) \cup \text{rest}(s_2, \pi) \\ \text{rest}(s, \pi) &= s \quad \text{otherwise} \end{aligned}$$

Example

$$\begin{aligned} \text{rest}(\llbracket C(n : \mathbf{dyn}) \rrbracket \cup \llbracket 0 \rrbracket, C) &= \text{rest}(\llbracket C(n : \mathbf{dyn}) \rrbracket, C) \cup \text{rest}(\llbracket 0 \rrbracket, C) \\ &= \perp \cup \llbracket 0 \rrbracket = \llbracket 0 \rrbracket \end{aligned}$$

After matching the C arm, the remainder shape for the else branch is $\llbracket 0 \rrbracket$.

Pattern Matching (Formalization)

Non Termination 1

Consider fibonacci number:

```
1 staged module Staged with
2   fun fib(n) = if n is
3     1 then 1
4     2 then 1
5     n then fib(n - 1) + fib(n - 2)
```

mls

With $n \mapsto \llbracket \text{dyn} \rrbracket$, every branch is viable. The recursive call $\text{fib}(n - 1)$ has argument shape $\llbracket \text{dyn} \rrbracket$, so shape propagation recurses forever, even though the original program terminates for valid inputs.

Pattern Matching (Formalization)

Solution: Stub Insertion

Before propagating into `fib(dyn)`, we **pre-insert a stub** into the cache:

Cache (stub inserted)

Function	Shapes	Symbol	Block	Returned Shape
<code>fib</code>	<code>(dyn,)</code>	<code>fib_Dyn</code>	<i>stub</i>	<code>dyn</code>

Now when propagating the `n` then `fib(n - 1) + fib(n - 2)` arm:

1. `fib(n - 1)` has argument shape `[[dyn]]` — matches the existing stub → rewritten to `fib_Dyn(n - 1)`.
2. Return shape `[[dyn]]` is read from the stub; recursion stops.
3. `fib(n - 2)` likewise hits the stub → rewritten to `fib_Dyn(n - 2)`.
4. Result shape of the arm: `[[dyn]] + [[dyn]] = [[dyn]]`.

Pattern Matching (Formalization)

The completed specialised function will be

```
1 fun fib_Dyn(n) = if n is
2   1 then 1
3   2 then 1
4   n then fib_Dyn(n - 1) + fib_Dyn(n - 2)
```

mls

Cache (stub filled)

Function	Shapes	Symbol	Block	Returned Shape
fib	(dyn,)	fib_Dyn	if n is ...	dyn

Remark. Stub insertion only guarantees termination of the first stage (shape propagation) when **the original program terminates**.

Pattern Matching (Formalization)

Non Termination 2

Consider a function that scans an array for a zero:

```
1 staged module Staged with
2   fun f(xs, i) =
3     if xs.(i) is
4       0 then i
5       v then f(xs, i + 1)
```

mls

With $xs \mapsto \llbracket \text{dyn} \rrbracket$ and $i \mapsto \llbracket 0 \rrbracket$ on the first call, $xs.(0)$ is $\llbracket \text{dyn} \rrbracket$ — both branches viable.

The recursive call has $i \mapsto \llbracket 1 \rrbracket$, then $\llbracket 2 \rrbracket$, ... — Shape Propagation diverges!

Pattern Matching (Formalization)

Solution: Context Decay

Before processing the branches of $\text{if } xs.(i) \text{ is}$, since the scrutinee has shape $\llbracket \text{dyn} \rrbracket$, we **decay** the context: every path that influenced the scrutinee is refined to $\llbracket \text{dyn} \rrbracket$.

Here $xs.(i)$ depends on xs and i , so both are decayed to $\llbracket \text{dyn} \rrbracket$ before entering the branches.

$$\begin{aligned} \text{decay}(\Gamma, p, s) &= \Gamma \quad \text{if } p \neq x \text{ for some variable } x \text{ or } s \neq \mathbf{dyn} \\ \text{decay}(\Gamma, x, \mathbf{dyn}) &= \Gamma \cdot \left[\overline{p_i \mapsto \mathbf{dyn}} \right]_{p_i \in \text{alias}(\text{clos}(x))} \end{aligned}$$

With $i \mapsto \llbracket \text{dyn} \rrbracket$ after decay, $i + 1 \mapsto \llbracket \text{dyn} \rrbracket$ too. The recursive call always hits a stable shape — propagation terminates.

Pattern Matching (Formalization)

Combining filter, rest, dead branch elimination and decay yields the following formalization:

$$\frac{\frac{\overline{\Gamma_0 \cdot (p \mapsto s_{i'})} \vdash b_i \rightsquigarrow b_{i'} \Rightarrow \Gamma_i, s_{i''}}{i=1..n, s_i \neq \perp} \quad \Gamma_0 \bowtie \overline{\Gamma_i - \Gamma_0}^{i=1..n, s_i \neq \perp} \vdash b_r \rightsquigarrow b_{r'} \Rightarrow \Gamma', s}{\frac{\Gamma \vdash p \Rightarrow s_0 \quad \Gamma_0 = \text{decay}(\Gamma, p, s_0) \quad \overline{s_{i'} = \text{filter}(s_{i-1}, \pi_i) \quad s_i = \text{rest}(s_{i-1}, \pi_i)}^n}{\Gamma \vdash \text{if } p \text{ is } \overline{\pi_i} \text{ then } \overline{b_i}^n; b_r \rightsquigarrow \text{if } p \text{ is } \overline{\pi_i} \text{ then } \overline{b_{i'}}^{i=1..n, s_i \neq \perp}; b_{r'} \Rightarrow s}}$$

Staged Class

Similar to staging module, we can stage classes as well, where we specializes on its method, with the main difference being dynamic dispatching `x.f()`.

```
1  staged class B1(val y) with
2    fun call(x) = x + 2 + y
3  staged class B2(val y) with
4    fun call(x) = x + y
5  staged module M with
6    fun twice(f, x) = f.call(f.call(x))
7    fun pick(x, y, b) = if b then x else y
8    fun f(b) =
9      let m = pick(new B1(2), new B2(3), b)
10     twice(m, 5)
```

Staged Class

Trace A: $f(\text{dyn})$

Specialise f with $b \mapsto \llbracket \text{dyn} \rrbracket$.

```
1 fun f(b) = mls  
2   let m = pick(new B1(2), new  
3     B2(3), b)  
   twice(m, 5)
```

f's ctx

$b \mapsto \llbracket \text{dyn} \rrbracket$

Staged Class

Trace A: $f(\text{dyn})$

Specialise f with $b \mapsto \llbracket \text{dyn} \rrbracket$.

```
1 fun f(b) = mls  
2   let m = pick(new B1(2), new  
   B2(3), b)  
3   twice(m, 5)
```

f 's ctx

$b \mapsto \llbracket \text{dyn} \rrbracket$

$m \mapsto \llbracket \perp \rrbracket$

1. let m : extend ctx with $m \mapsto \llbracket \perp \rrbracket$

Staged Class

Trace A: $f(\text{dyn})$

Specialise f with $b \mapsto \llbracket \text{dyn} \rrbracket$.

```
1 fun f(b) = mls  
2   let m = pick(new B1(2), new  
   B2(3), b)  
3   twice(m, 5)
```

f 's ctx

$b \mapsto \llbracket \text{dyn} \rrbracket$

$m \mapsto \llbracket \perp \rrbracket$

1. let m : extend ctx with $m \mapsto \llbracket \perp \rrbracket$
2. Call $\text{pick}(\text{new } B1(2), \text{new } B2(3), b)$:
argument shapes are $\llbracket B1(2) \rrbracket$,
 $\llbracket B2(3) \rrbracket$, $\llbracket \text{dyn} \rrbracket$. Recurse into pick
with a fresh ctx $\rightarrow$ **go to Trace B**

Staged Class

Trace B: pick(B1(2), B2(3), dyn)

```
1 fun pick(x, y, b) =  
2   if b then x  
3   else y
```

mls

pick's ctx

$x \mapsto \llbracket B1(2) \rrbracket$

$y \mapsto \llbracket B2(3) \rrbracket$

$b \mapsto \llbracket \text{dyn} \rrbracket$

Staged Class

Trace B: `pick(B1(2), B2(3), dyn)`

```
1 fun pick(x, y, b) = mls
2   if b then x
3   else y
```

pick's ctx

$x \mapsto \llbracket B1(2) \rrbracket$

$y \mapsto \llbracket B2(3) \rrbracket$

$b \mapsto \llbracket \text{dyn} \rrbracket$

1. if b: b is $\llbracket \text{dyn} \rrbracket$, so both branches viable

Staged Class

Trace B: `pick(B1(2), B2(3), dyn)`

```
1 fun pick(x, y, b) =  
2   if b then x  
3   else y
```

mls

pick's ctx

$x \mapsto \llbracket B1(2) \rrbracket$

$y \mapsto \llbracket B2(3) \rrbracket$

$b \mapsto \llbracket dyn \rrbracket$

1. if b: b is $\llbracket dyn \rrbracket$, so both branches viable
2. then arm returns x, shape = $\llbracket B1(2) \rrbracket$

Staged Class

Trace B: `pick(B1(2), B2(3), dyn)`

```
1 fun pick(x, y, b) =  
2   if b then x  
3   else y
```

mls

pick's ctx

$x \mapsto \llbracket B1(2) \rrbracket$

$y \mapsto \llbracket B2(3) \rrbracket$

$b \mapsto \llbracket \text{dyn} \rrbracket$

1. if b: b is $\llbracket \text{dyn} \rrbracket$, so both branches viable
2. then arm returns x, shape = $\llbracket B1(2) \rrbracket$
3. else arm returns y, shape = $\llbracket B2(3) \rrbracket$

Staged Class

Trace B: `pick(B1(2), B2(3), dyn)`

```
1 fun pick(x, y, b) =  
2   if b then x  
3   else y
```

mls

pick's ctx

$x \mapsto \llbracket B1(2) \rrbracket$

$y \mapsto \llbracket B2(3) \rrbracket$

$b \mapsto \llbracket \text{dyn} \rrbracket$

1. if b: b is $\llbracket \text{dyn} \rrbracket$, so both branches viable
2. then arm returns x, shape = $\llbracket B1(2) \rrbracket$
3. else arm returns y, shape = $\llbracket B2(3) \rrbracket$
4. Result shape: $\llbracket B1(2) \cup B2(3) \rrbracket$.
Cached as pick1. Return to Trace A.

Staged Class

Trace A (continued): back in f

```
1 fun f(b) = m!s  
2   let m = pick(new B1(2), new  
3     B2(3), b)  
   twice(m, 5)
```

f's ctx

$b \mapsto \llbracket \text{dyn} \rrbracket$

$m \mapsto \llbracket B1(2) \cup B2(3) \rrbracket$

Staged Class

Trace A (continued): back in f

```
1 fun f(b) = m!s  
2   let m = pick(new B1(2), new  
3     B2(3), b)  
   twice(m, 5)
```

f's ctx

$b \mapsto \llbracket \text{dyn} \rrbracket$

$m \mapsto \llbracket B1(2) \cup B2(3) \rrbracket$

1. Bind m to pick's returned shape =
 $\llbracket B1(2) \cup B2(3) \rrbracket$

Staged Class

Trace A (continued): back in f

```
1 fun f(b) = m!s  
2   let m = pick(new B1(2), new  
3     B2(3), b)  
3   twice(m, 5)
```

f's ctx

$b \mapsto \llbracket \text{dyn} \rrbracket$

$m \mapsto \llbracket B1(2) \cup B2(3) \rrbracket$

1. Bind m to pick's returned shape = $\llbracket B1(2) \cup B2(3) \rrbracket$
2. $\text{twice}(m, 5): \rightarrow$ **go to Trace C** and specialise as `twice1`

Staged Class

Trace C: `twice({B1(2), B2(3)}, 5)`

```
1 fun twice(f, x) =  
2   let tmp  
3   tmp = f.call(x)  
4   f.call(tmp)
```

mls

twice's ctx

$f \mapsto \llbracket B1(2) \cup B2(3) \rrbracket$

$x \mapsto \llbracket 5 \rrbracket$

Recall:

```
1 class B1(y) with  
2   fun call(x) = x + 2 + y  
3 class B2(y) with  
4   fun call(x) = x + y
```

mls

Staged Class

Trace C: `twice({B1(2), B2(3)}, 5)`

```
1 fun twice(f, x) =  
2   let tmp  
3   tmp = f.call(x)  
4   f.call(tmp)
```

mls

Recall:

```
1 class B1(y) with  
2   fun call(x) = x + 2 + y  
3 class B2(y) with  
4   fun call(x) = x + y
```

mls

`twice's ctx`

$f \mapsto \llbracket B1(2) \cup B2(3) \rrbracket$

$x \mapsto \llbracket 5 \rrbracket$

$tmp \mapsto \llbracket \perp \rrbracket$

1. let tmp: extend ctx with $tmp \mapsto \llbracket \perp \rrbracket$

Staged Class

Trace C: `twice({B1(2), B2(3)}, 5)`

```
1 fun twice(f, x) =  
2   let tmp  
3   tmp = f.call(x)  
4   f.call(tmp)
```

mls

Recall:

```
1 class B1(y) with  
2   fun call(x) = x + 2 + y  
3 class B2(y) with  
4   fun call(x) = x + y
```

mls

twice's ctx

$f \mapsto \llbracket B1(2) \cup B2(3) \rrbracket$

$x \mapsto \llbracket 5 \rrbracket$

$tmp \mapsto \llbracket \perp \rrbracket$

2. `f.call(x)` on union $\rightarrow$ **split by class**:

► $f = \llbracket B1(2) \rrbracket$:

B1.call's ctx

$this \mapsto \llbracket B1(2) \rrbracket$

$x \mapsto \llbracket 5 \rrbracket$

cache `f.call1()` in B1 $\rightarrow \llbracket 9 \rrbracket$

► $f = \llbracket B2(3) \rrbracket$: cache `f.call1()` in B2 $\rightarrow \llbracket 8 \rrbracket$

Staged Class

Trace C: `twice({B1(2), B2(3)}, 5)`

```
1 fun twice(f, x) =  
2   let tmp  
3   tmp = f.call(x)  
4   f.call(tmp)
```

mls

Recall:

```
1 class B1(y) with  
2   fun call(x) = x + 2 + y  
3 class B2(y) with  
4   fun call(x) = x + y
```

mls

twice's ctx

$f \mapsto \llbracket B1(2) \cup B2(3) \rrbracket$

$x \mapsto \llbracket 5 \rrbracket$

$tmp \mapsto \llbracket 9 \cup 8 \rrbracket$

3. `tmp = f.call(x)`: bind tmp to $\llbracket 9 \cup 8 \rrbracket$

Staged Class

Trace C (continued): second call

```
1 fun twice(f, x) = mls  
2   let tmp  
3   tmp = f.call(x)  
4   f.call(tmp)
```

Recall:

```
1 class B1(y) with fun call(x) mls  
  = x + 2 + y  
2 class B2(y) with fun call(x) = x  
  + y
```

twice's ctx

$f \mapsto \llbracket B1(2) \cup B2(3) \rrbracket$

$x \mapsto \llbracket 5 \rrbracket$

$tmp \mapsto \llbracket 9 \cup 8 \rrbracket$

4. $f.call(tmp)$ with tmp in $\llbracket 9 \cup 8 \rrbracket$: split again, body kept as code
- ▶ $f = \llbracket B1 \rrbracket$: cache $f.call2(x) = x + 2 + 2$ in $B1 \rightarrow \llbracket 13 \cup 12 \rrbracket$
 - ▶ $f = \llbracket B2 \rrbracket$: cache $f.call2(x) = x + 3$ in $B2 \rightarrow \llbracket 12 \cup 11 \rrbracket$

Staged Class

Resulting Cache

After propagation, the residual M and the residual classes contain:

```
1  module M with mls
2    fun f(b) =
3      let m, tmp1, tmp2
4      tmp1 = new B1(2)
5      tmp2 = new B2(3)
6      m = pick1(tmp1, tmp2,
7               b)
7    twice1(m)
8    fun pick1(x, y, b) =
9      if b then new B1(2)
        else new B2(3)
```

```
1  fun twice1(f) = mls
2    let tmp
3    if f is
4      B1 then tmp = f.call1()
5      B2 then tmp = f.call1()
6    if f is
7      B1 then f.call2(tmp)
8      B2 then f.call2(tmp)
```

Staged Class

```
1  class B1(y) with mls
2    fun call1() = 9
3    fun call2(x) =
4      let tmp
5        tmp = x + 2
6        tmp + 2
7
8  class B2(y) with
9    fun call1() = 8
10   fun call2(x) = x + 3
```

Remark. If f 's shape included an **unstaged** class B3, we could not specialise the B3.call function without its Staged Block. The generated `twice1` would then include an else branch falling back to the original `f.call(...)`.

Motivation	1
Literature Survey	4
Overview	13
Staging	26
Shape Propagation	35
Inlining	71
Inlining	72
Printing Staged Block	74
Benchmarking	77
Bibliography	79

Inlining

A typical module after Dynamic Staging will look like the following.

Dynamic Staging Output

```
1 module M with mls
2   fun pow_Dyn_Lit1(x) =
3     let {tmp, tmp1}
4       x
5   fun pow_Dyn_Lit2(x) =
6     let {tmp, tmp1}
7       tmp = 1
8       tmp1 = pow_Dyn_Lit1(x)
9       *(x, tmp1)
```

```
1   fun pow_Dyn_Lit3(x) = mls
2     let {tmp, tmp1}
3     tmp = 2
4     tmp1 = pow_Dyn_Lit2(x)
5     *(x, tmp1)
```

Inlining

During **the second stage of compilation**, we utilize the MLscript compiler's existing inlining capabilities to inline function calls, as recursion has been eliminated during dynamic staging.

```
1  let x, inlinedVal, tmp1, x1, inlinedVal1, tmp11, x2, inlinedVal2;
2  x = 2;
3  x1 = x;
4  x2 = x1;
5  inlinedVal2 = x2;
6  tmp11 = inlinedVal2;
7  inlinedVal1 = x1 * tmp11;
8  tmp1 = inlinedVal1;
9  inlinedVal = x * tmp1;
10 return inlinedVal
```

Js JavaScript

Motivation 1

Literature Survey 4

Overview 13

Staging 26

Shape Propagation 35

Inlining 71

Printing Staged Block 74

 Printing Staged Block 75

Benchmarking 77

Bibliography 79

Printing Staged Block

After all the specialized functions are completed, we need to write the functions to a new file.

```
1 staged module M with
2   val funCache = new Map([
3     ["f1", ...],
4     ["f2", ...],
5     ["g1", ...]
6   ])
```

mls

```
1 module M with
2   fun f1(x, y) = ...
3   fun f2() = ...
4   fun g1() = ...
```

mls

Printing Staged Block

This is a similar process to staging, but done in reverse.

As before, we need extra care when handling symbols.

```
1 import "../M.mls"
```

mls

Motivation	1
Literature Survey	4
Overview	13
Staging	26
Shape Propagation	35
Inlining	71
Printing Staged Block	74
Benchmarking	77
Model-View-Projection transformation	78
Bibliography	79

Model-View-Projection transformation

Used in 3D rendering

$$\vec{v} = MVP \cdot \vec{u}$$

Evaluate matrix multiplications when the matrices are known values.

Benchmark: Transform 16k random coordinates using a known transformation matrices

Time

Unstaged: 2s

Time: about 2x (from 2s to 1s, eh...)

Code size: don't worry about it, we'll just do lifting

Bibliography

- [1] W. Taha, “A gentle introduction to multi-stage programming,” in *Domain-Specific Program Generation: International Seminar, Dagstuhl Castle, Germany, March 23-28, 2003. Revised Papers*, 2004, pp. 30–50.
- [2] L. Parreaux, A. Shaikhha, and C. E. Koch, “Quoted staged rewriting: a practical approach to library-defined optimizations,” in *Proceedings of the 16th ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences*, 2017, pp. 131–145.
- [3] A. Shali and W. R. Cook, “Hybrid partial evaluation,” in *Proceedings of the 2011 ACM international conference on Object oriented programming systems languages and applications*, 2011, pp. 375–390.

- [4] K. Swadi, W. Taha, O. Kiselyov, and E. Pasalic, “A monadic approach for avoiding code duplication when staging memoized functions,” in *Proceedings of the 2006 ACM SIGPLAN symposium on Partial evaluation and semantics-based program manipulation*, 2006, pp. 160–169.